



34. Rule-Based Classifiers

"If it's in the rule, it's in the class"

[Previous Section:](#)

 [33. Convex Optimization Problems](#)

[Contents:](#)

[1. Rule-Based Classifiers – The Overview](#)

[2. Sequential Covering Algorithm](#)

[3. General-to-Specific vs Specific-to-General](#)

[4. The RIPPER Algorithm](#)

[5. RIPPER Algorithm by Hand](#)

[6. RIPPER Algorithm - The Pruning Phase](#)

[7. RIPPER Algorithm with Python](#)

[Summary](#)

[References](#)

Rule-Based Classifiers feel familiar. Because before Machine Learning became what it is today, this was how we *naturally* tried to make computers intelligent. We wrote rules. Clear, logical, step-by-step instructions:

"If this happens → then assign this class."; "If that condition is true → then follow this path."

Simple. Structured. Understandable. And in many ways, that idea never disappeared.

When you first learned about Decision Trees, you were already thinking in rules—just in a slightly different form. Each path from the root to a leaf was essentially a rule:

"If Feature A $\leq$ threshold... **and** Feature B $>$ threshold... then Class = X"

So even though Decision Trees look like diagrams underneath, they are nothing more than a collection of rules. And this is exactly where Rule-Based Classifiers come in. Instead of representing logic as a tree we flatten it. We extract the logic and express it directly:

- A set of **IF-THEN rules** → A structured list of decisions → A system that says: **"If the rule matches, the class follows."**

But here's the important shift. Decision Trees are **implicit rule systems**. The rules are hidden inside the tree structure.

Rule-Based Classifiers are **explicit rule systems**. The rules are written out, visible, and directly interpretable.

And that difference matters more than it seems. Because now, you are no longer navigating a tree. You are reasoning through a rule set.

So why do we even need this? Because sometimes, trees become messy.

As datasets grow more complex, Decision Trees can become deep, hard to read, and difficult to interpret. Rules, on the other hand, give us something cleaner:

- Modular logic (each rule stands on its own)
- Easier interpretation (no need to trace paths)
- Better control over decision-making

It's like moving from a tangled map to a checklist of conditions.

So when you think about Rule-Based Classifiers, don't think of them as something completely new. Think of them as the **unwrapped version of Decision Trees**. Same philosophy. Same logic. Just a different—and often clearer representation.

1. Rule-Based Classifiers – The Overview

A Rule-Based Classifier is one of those models that feels almost too straightforward at first. Because instead of diving into heavy mathematics, or transforming data into some abstract space, it does something very human: ***It follows rules.***

Think about it like this. Instead of asking the model to *learn a complex function...* we ask it a much simpler question:

■ ***"If these conditions are true... what should the answer be?"***

And from that idea, everything else follows. A Rule-Based Classifier is a supervised learning model that makes decisions using a collection of explicit **IF-THEN rules**. Each rule describes a situation and the outcome that should follow.

So rather than saying: ***"Let me compute something complicated..."*** → The model says: ***"Let me check if this condition matches."*** → And if it does, the class is assigned.

Now here's why this matters. Every prediction the model makes can be traced back, not to some hidden computation, but to a rule you can actually read. Not interpret. Not approximate. Read. And that makes these models extremely interpretable. Because at any moment, you can ask: "Why did the model make this decision?" → And the answer will be: ***"Because this rule was satisfied."***

At their core, rule-based classifiers describe relationships in data using logical statements learned from examples. And if that sounds familiar, it should. Because you've already seen this idea before—inside Decision Trees.

Every path from the root of a Decision Tree to a leaf node can be rewritten as a rule:

```
"If condition A... and condition B... and condition C... then Class = X"
```

Which means a Decision Tree is not just a tree. It is a collection of rules, hiding in a different form.

Let's make that concrete. Example:

```
IF (humidity = high) AND (outlook = sunny) THEN play = no
```

Now break it down:

- Each condition in the IF part? → That's a decision node in a tree
- The final outcome in the THEN part? → That's the leaf node

And here's the key: The prediction only happens when **all conditions are satisfied**. Not partially. Not approximately. Completely.

Now we give names to these parts. Because in rule-based systems, structure matters.

- The IF part is called the **Rule Antecedent** → It defines the conditions
- The THEN part is called the **Rule Consequent** → It defines the outcome

So every rule becomes: **Antecedent (conditions) → Consequent (prediction)**

Simple. Clean. Precise. But now we reach the more interesting question. How does the model actually *get* these rules? Because unlike writing them manually. Here, the machine must learn them from data.

And there are two fundamentally different ways this can happen. Two philosophies. Two approaches: **Indirect methods and Direct methods**. Both aim to uncover patterns. Both produce rules. But the way they arrive there is completely different. And that difference? That's where things start to get interesting.

(1) Indirect Methods (Model-Based Rule Extraction)

Let's talk about the first way machines learn rules. But here's the twist. In this approach... the model is not *trying* to learn rules. At least not directly.

Indirect methods start by doing something very familiar: Train a traditional classification model. Only after that, we step back and ask: **"Can we turn what this model learned... into rules?"**

So the rules are not the goal. They are... a byproduct.

The most classic example? A Decision Tree. You train the tree like usual. It grows, splits, and organizes the data into a hierarchy of decisions. But once it's done, something interesting happens. Every path from the root to a leaf becomes a rule:

```
If this condition... and this condition... and this condition... then Class = X"
```

So instead of a tree, you now have a rule set. Same knowledge. Different representation.

And this gives us a very nice advantage. You get the predictive strength of the original model, while still ending up with something interpretable.

But there's a catch. The quality of your rules depends entirely on the model you trained.

- If the tree is deep, your rules become long and messy.
- If the tree is unstable, your rules become inconsistent.

So while this method is powerful, it inherits both the strengths, and the weaknesses of the underlying model.

(2) Direct Methods (Data-Driven Rule Induction)

Now we flip the idea completely. **What if we don't train a model first? What if the rules themselves are the model?**

That's exactly what direct methods do. They go straight to the data and start searching for patterns in the form of rules. No intermediate structure. No hidden representation. Just: "What conditions lead to this class?"

For example, the model might scan the dataset and discover:

"Whenever Feature A = this... and Feature B = that... the outcome is usually X"

And from that, it builds a rule. Not extracted. Not converted. But **discovered directly**.

Because of this, direct methods tend to focus heavily on simplicity. They try to produce rules that are easy to understand, easy to interpret, easy to validate

But like everything in Machine Learning, there's a trade-off. Sometimes, by focusing too much on simplicity, they sacrifice predictive performance compared to more complex models.

The Real Difference (And Why It Matters)

At first glance, both approaches give you rules. So what's the real difference? It's not just *how* rules are created. It's *how the model searches for them*.

Indirect methods (like Decision Trees) explore the problem in a **breadth-first** way.

- They grow the entire structure level by level
- Multiple decision paths are developed at the same time
- Every root-to-leaf path becomes a potential rule

⇒ It's like building a full map, before extracting directions.

Direct methods, on the other hand, think very differently. They follow a **depth-first** approach.

- Focus on one rule at a time
- Refine it, improve it, finalize it
- Then move on to the next rule

⇒ It's like solving one case completely before starting another.

And this difference changes everything.

- Breadth-first → broader exploration, but less focused per rule
- Depth-first → more control, more refinement, but narrower at each step

Why Rule-Based Classifiers Still Matter

So after all of this, you might be wondering: ***"With all these advanced models... why do we still care about rules?"*** → Because sometimes, performance is not enough. Sometimes, you need to understand *why* a decision was made.

In domains like: Healthcare; Finance; Policy-making. A black-box model is not always acceptable. You don't just want predictions. You want explanations. And that's where rule-based classifiers shine. They give you a balance: Strong enough to make meaningful predictions; Simple enough to be understood and trusted.

Because at the end of the day, a model that explains itself is sometimes more valuable than one that is just slightly more accurate.

2. Sequential Covering Algorithm

Now instead of jumping straight into formulas and procedures, let's understand this the way we naturally would.

First, what is Sequential Covering? It is a rule-learning strategy where rules are generated **one at a time**, and each rule is used to "cover" a portion of the dataset. Once a rule explains some examples, those examples are removed, and the process continues on what remains.

Simple idea. But to really *feel* it, let's step into a story.

The Farm Analogy - Imagine a farmer managing a large farm full of horses. This farm is your dataset. Each horse is a data point. And the farmer has one goal: *Determine whether each horse is a mare or a stallion* → So what are the features X and Y ? They are just characteristics of the horses: **(Size, Color, Strength, Speed)**.

At the beginning, everything is mixed together. No structure. No separation. Just data.

- **Step 1: The Setup:** All horses are on the same plot. Mares and stallions... mixed together. Just like a raw dataset... before learning begins.
- **Step 2: Identify a Group:** The farmer starts observing carefully. He looks for a group of horses that share the same gender and have similar characteristics. Maybe he finds a cluster of mares in one area. They all have similar values of X and Y . So he forms a rule:

```
IF (X and Y fall within a certain range) THEN gender = mare
```

This rule now **explains** that group. In machine learning terms: the rule *covers* those examples.

- **Step 3: Remove Covered Examples:** Now comes the key move. The farmer removes those horses from the plot. Why? Because they are already explained. There's no need to

reconsider them.

In machine learning: Covered examples are removed from the dataset. So the next rule doesn't waste time on what is already known.

- **Step 4: Repeat the Process:** Now the farm is smaller. Fewer horses. Less complexity. The farmer repeats the process:
 - Find another group (R2)
 - Create a rule → Remove those horses
 - Then again: R3, R4, and so on

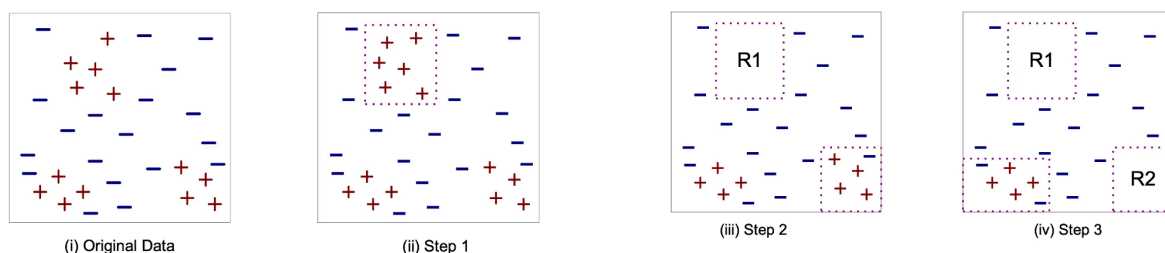
This continues until: All mares are covered. Then the farmer switches focus: Now handle stallions. Same process. Same logic. Until **every horse** is explained by at least one rule.

This is Sequential Covering. Not everything at once. But **one piece** at a time.

- **Step 5: Classify New Horses:** Now imagine a new horse arrives. The farmer doesn't panic. He already has rules. So he checks: *Which rule matches this horse's characteristics?*
 - If the horse fits R1 → assign that class
 - If not, check R2 → then R3 → and so on

The first matching rule determines the outcome. And just like that, the new horse is classified. This is exactly how a rule-based classifier works on unseen data.

(The below image is for reference only)



There's something very elegant about this approach. Because it aligns with how we naturally think. We don't try to understand everything at once. We break problems into smaller pieces, solve them, and move forward.

In practice, Sequential Covering shines when interpretability is critical. You need clear, human-readable decisions. You want to understand *why* a prediction was made. This is why algorithms like **CN2** and **RIPPER** are built on this idea. They don't just predict. They explain.

So in the end, Sequential Covering is not just about building rules. It's about building them **intelligently**. One step at a time. One pattern at a time. Until the entire dataset is understood.

3. General-to-Specific vs Specific-to-General

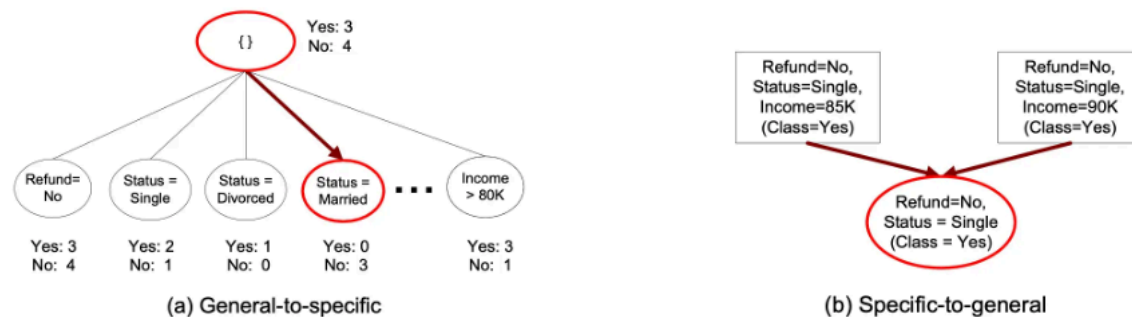
We reach a question that is more subtle than it looks. We already know *how* rules are learned. We already know *how* they are applied. But here's the deeper question: "**How do we search for the right rule... in the first place?**"

Because the space of possible rules is huge. So the real challenge is not just learning rules. It's **navigating the space of rules intelligently**.

And this is where two very different strategies appear → Two opposite ways of thinking: Start broad, then narrow down, or start narrow, then expand outward.

(The below image is for reference only)

Two common strategies



General-to-Specific (Start Big → Then Focus)

Let's begin with the more intuitive one. The general-to-specific approach starts with something almost meaningless. A rule so broad it applies to everything:

"IF (True) THEN class = c"

Which basically says: "Everything belongs to class c" → Clearly, that's not useful. But it's a starting point.

From here, the algorithm begins to **add conditions**. Each condition acts like a filter. Each step makes the rule more precise, more selective, more accurate

So the rule evolves like this:

```
IF (outlook = sunny) THEN play = no
IF (outlook = sunny) AND (humidity = high) THEN play = no
```

With every added condition, the rule covers fewer examples. But makes fewer mistakes

And this is the key idea: We are **shrinking the rule** until it becomes reliable. At each step, the algorithm asks: "Which condition improves this rule the most?". And it chooses based on some measure: Accuracy; Information gain; or Confidence.

So instead of guessing, it refines. Carefully. Iteratively. Intentionally.

What defines this approach? Starts broad, then narrows. Focuses on reducing false positives. Produces compact, clean rules. Works very well with pruning (removing unnecessary conditions)

This is why it fits perfectly with Sequential Covering. Because it's efficient. It finds strong rules quickly, and moves on.

Specific-to-General (Start Small → Then Expand)

Now let's flip everything. Instead of starting with everything. We start with almost nothing.

The specific-to-general approach begins with a rule that is **too specific**. So specific, it might only describe a single example:

```
IF (outlook = sunny) AND (humidity = high) AND (temperature = hot) THEN  
play = no
```

This rule is *perfect*, but only for one case. So what do we do? We start **removing conditions**. One by one. And each removal makes the rule broader, more flexible, and more general

So the rule becomes:

```
IF (outlook = sunny) AND (humidity = high) THEN play = no
```

Now it applies to more examples.

And that's the core idea: We are **expanding the rule** until it captures a meaningful pattern. But carefully. Because if we remove too much, the rule becomes inaccurate.

This approach is often tied to example-driven learning. You start with a real data point and grow a rule around it.

What defines this approach? Starts narrow, then expands. Focuses on reducing false negatives. Builds rules from actual examples. Can capture subtle, local patterns

But there's a risk. If you don't generalize enough, the rule stays too specific and fails to generalize to new data.

So now you see it. Both approaches are doing the same thing: Balancing **coverage** and **accuracy**. But they attack the problem from opposite sides.

General-to-specific: **"Start wide... then eliminate mistakes"** → Specific-to-general: **"Start precise... then include more cases"**

And in practice? We rarely stick to just one. Real systems often blend both ideas. Because the goal is not just to find rules, but to find rules that are: Accurate → Interpretable → General enough to handle new data

So in the end, Rule learning is not just about logic. It's about **search**. And whether you start from everything or from a single example. You're always trying to answer the same question: **"What is the simplest rule... that still tells the truth?"**

4. The RIPPER Algorithm

So far, we've seen how rule-based classifiers follow a simple philosophy: **Learn a rule** → **Remove what it explains** → **Repeat**.

That's the idea of **sequential covering**. But here's the problem... **what stops the model from getting too greedy?** What stops it from creating rules that perfectly fit the training data... but completely fail in the real world? → This is where **RIPPER** comes in.

RIPPER (Repeated Incremental Pruning to Produce Error Reduction) is not just about building rules. It's about building rules, **then questioning them**. Think of it like this:

- First → *"Let me find a pattern that works."*
- Then → *"Wait... am I overfitting?"* → *"Let me simplify this before I trust it."*

That second step? That's what makes RIPPER powerful.

At a high level, RIPPER still follows sequential covering: Learn one rule → Remove covered examples → Repeat. But it adds something critical: **Separation of learning and validation**

Instead of using all data blindly, RIPPER splits it into:

- **Growing Set** → Used to build rules
- **Pruning Set** → Used to *challenge* those rules

How RIPPER Decides What to Add - Now comes the real question:

┆ *When building a rule... how does RIPPER decide what condition is "good"?*

It doesn't guess. It measures. RIPPER evaluates candidate conditions using an **information gain-based metric**. When we refine a rule R into a new rule R' , the gain is defined as:

$$Gain(R', R) = s \left(\log_2 \frac{N'_+}{N'} - \log_2 \frac{N_+}{N} \right)$$

Let's decode it step by step:

- R : the original rule
- R' : the rule after adding a new condition
- N, N' : total examples covered before and after refinement
- N_+, N'_+ : number of **true positives** covered
- s : number of true positives still covered after refinement

This formula is not just math, it encodes a very specific idea: *"Does this new condition make the rule **more focused on the right cases?**"*

It rewards conditions that:

- Increase the **purity** of the rule (more positives, fewer negatives)
- While still keeping **enough coverage** (not becoming too narrow)

So RIPPER is constantly balancing **Precision** → "Am I correct?" and **Coverage** → "Am I useful?"

When RIPPER grows a rule, it is not trying to be perfect immediately. It is trying to gradually **increase confidence** while avoiding becoming **too specific too early**.

Because a rule that is *too perfect* is often just **memorizing**. And RIPPER? It doesn't want rules that *look good* → It wants rules that **actually work**.

This is not just a technical trick. This is a philosophy:

┃ *"Don't trust a rule just because it fits the data... Trust it because it survives scrutiny."*

Step 1: Growing a Rule (Be Greedy, but Smart)

We start from nothing: Empty rule.

Then we keep adding conditions one at a time. Each time choosing what improves the rule the most

But how do we measure "improvement"? RIPPER uses an **information gain-style idea**: Does this condition increase the proportion of positive examples?

A common form during growth is:

$$p \left(\log \frac{p}{t} - \log \frac{P}{T} \right)$$

Where:

- p : positives covered by the rule
- t : total covered
- P : total positives in dataset
- T : total examples

Interpretation (this matters more than the formula): We want rules that are **pure** (mostly positive). But also **useful** (cover enough examples). So the rule keeps growing until it becomes "good enough" or nothing improves it anymore.

Step 2: Pruning the Rule (Now Be Skeptical)

This is where RIPPER becomes different. After growing a rule, it immediately tries to **simplify it**. Using the **pruning set**, it asks: *"Do I really need all these conditions?"*

A common pruning metric is:

$$\frac{p - n}{p + n}$$

Or smoother variants:

$$\frac{2p}{p + n} - 1 \quad \text{or} \quad \frac{p + 1}{p + n + 2}$$

Where:

- p : positives covered

- n : negatives covered

If $p + n = 0 \rightarrow$ score defaults to 0.5 $\rightarrow$ Why? Because no coverage = no information $\rightarrow$ So we assign a neutral value

Pruning removes conditions that don't improve accuracy or only help memorize noise. So instead of: *"Perfect rule for training data"* $\rightarrow$ We get: *"Reliable rule for unseen data"*

Step 3: Cover and Repeat

Once a rule is finalized. Add it to the rule set. Remove the examples it covers

Then repeat the process: **Grow** $\rightarrow$ **Prune** $\rightarrow$ **Add** $\rightarrow$ Until no useful rules can be found

Step 4: Optimization (RIPPER Doesn't Trust Itself, Again)

Even after building rules, RIPPER goes back and asks: "Can I do better?"

So it may re-grow rules, re-prune them, replace weak rules. This is rare in simpler algorithms. Most models stop after learning. RIPPER says: *"Learning once is not enough."*

Step 5: Rule Deletion (Keep Only What Matters)

Finally, some rules are simply not worth keeping. So RIPPER removes redundant rules $\rightarrow$ rules that don't improve performance. This results in a **compact** rule set, easier to interpret and less noise

If we zoom out, RIPPER is not just a sequence of steps. It's a cycle of **confidence and doubt**:

1. Learn aggressively (Grow)
2. Question aggressively (Prune)
3. Re-evaluate everything (Optimize)
4. Remove what doesn't matter (Delete)

Why RIPPER Works is because most rule learners fail in one of two ways:

- Too simple $\rightarrow$ miss patterns
- Too complex $\rightarrow$ overfit

RIPPER balances both:

- **Greedy growth** $\rightarrow$ captures patterns
- **Pruning** $\rightarrow$ avoids overfitting
- **Optimization** $\rightarrow$ refines decisions
- **Deletion** $\rightarrow$ keeps things clean

RIPPER follows a very human way of learning:

- First, you **notice patterns**

- Then, you **realize you might be wrong**
- Then, you **simplify your understanding**
- And finally, you **keep only what actually works**

That's why it scales well. That's why it handles noise. And that's why it remains one of the most practical rule-based classifiers.

5. RIPPER Algorithm by Hand

Alright, now this is where things get interesting. We're not just *describing* RIPPER anymore. We're actually **thinking like it**. Let's walk through this example my way - the **I.R.I.S way** - step by step, but more importantly, *decision by decision*

We will use this simple Play Tennis dataset:

Serial No.	Outlook	Temperature	Humidity	Windy	Play
1	Sunny	Hot	High	FALSE	No
2	Sunny	Hot	High	TRUE	No
3	Overcast	Hot	High	FALSE	Yes
4	Rainy	Mild	High	FALSE	Yes
5	Rainy	Cool	Normal	FALSE	Yes
6	Rainy	Cool	Normal	TRUE	No
7	Overcast	Cool	Normal	TRUE	Yes
8	Sunny	Mild	High	FALSE	No
9	Sunny	Cool	Normal	FALSE	Yes
10	Rainy	Mild	Normal	FALSE	Yes
11	Sunny	Mild	Normal	TRUE	Yes
12	Overcast	Mild	High	TRUE	Yes
13	Overcast	Hot	Normal	FALSE	Yes
14	Rainy	Mild	High	TRUE	No

Step 0: Initialization — Where Do We Even Begin?

We start with nothing: Rule Set: **RS = {}**

Now here's a very important decision RIPPER makes: It **does NOT start with the majority class**. Instead, it starts with the **minority class**.

Why? Because mistakes on rare cases are more costly → Fixing them early reduces overall error

In this dataset: $Play = Yes \rightarrow 9$ (majority); $Play = No \rightarrow 5$ (minority)

So we begin with: *Target : Play = No*

Step 1: Focus Only on What We Care About

Before building rules, RIPPER isolates the target: "What does *No* actually look like?"

We extract all negative examples:

Serial No.	Outlook	Temperature	Humidity	Windy	Play
1	Sunny	Hot	High	FALSE	No
2	Sunny	Hot	High	TRUE	No
6	Rainy	Cool	Normal	TRUE	No
8	Sunny	Mild	High	FALSE	No
14	Rainy	Mild	High	TRUE	No

Total dataset: 14; Total negatives (No): 5 → So initially: $N^+ = 5$ (target class = *No*); $N = 14$

Step 2: Growing the First Rule — The First Guess

Now RIPPER asks: *“What simple condition already captures the pattern?”* → We test candidates.

- **Candidate 1: Outlook = Sunny:** Covers: 5 examples → True positives (No): 3

$$Gain = 3 \left(\log_2 \frac{3}{5} - \log_2 \frac{5}{14} \right)$$

→ Pretty strong signal

- **Candidate 2: Outlook = Overcast:** Covers: 4 → True positives: 0

$$Gain = 0$$

→ Completely useless

- **Candidate 3: Outlook = Rainy:** Covers: 5 → True positives: 2

$$Gain = 2 \left(\log_2 \frac{2}{5} - \log_2 \frac{5}{14} \right)$$

→ Some signal, but weaker

- **Decision:** RIPPER picks the condition with **highest gain: Outlook = Sunny**

Step 3: Refining the Rule — “Good, but Not Good Enough”

Now we zoom into: Only examples where **Outlook = Sunny**

Serial No.	Outlook	Temperature	Humidity	Windy	Play
1	Sunny	Hot	High	FALSE	No
2	Sunny	Hot	High	TRUE	No
8	Sunny	Mild	High	FALSE	No
9	Sunny	Cool	Normal	FALSE	Yes

11	Sunny	Mild	Normal	TRUE	Yes
----	-------	------	--------	------	-----

We see: **3 No; 2 Yes** → Still mixed → not acceptable. So RIPPER asks:

“What condition separates these cleanly?”

Testing: Humidity

- *Humidity = High* → **3 No, 0 Yes**
- *Humidity = Normal* → **0 No, 2 Yes**

→ Perfect separation

Final Rule (1)

IF Outlook = Sunny AND Humidity = High → Play = No

This is **pure**. No mistakes. No ambiguity. So RIPPER says: *“This rule... I trust.”* → Add to *RS*.

Step 4: Cover and Remove — Commit to the Decision

Now we remove all examples covered by **Rule (1)**. Remaining “No” examples:

Serial No.	Outlook	Temperature	Humidity	Windy	Play
6	Rainy	Cool	Normal	TRUE	No
14	Rainy	Mild	High	TRUE	No

Step 5: Growing the Next Rule — Patterns Get Smaller

Now the dataset is smaller. RIPPER repeats the same thinking: *“What condition now explains the remaining No cases?”*

Testing conditions

- *Outlook = Rainy* → still mixed; *Windy = TRUE* → **2 No, 0 Yes**

→ Perfect separation again

Final Rule (2)

IF Outlook = Rainy AND Windy = TRUE → Play = No

Add to *RS*. Remove covered examples.

Now: All “No” cases are covered → So RIPPER moves on.

Step 6: Switch Perspective — Now Learn “Yes”

Now we flip the problem: Target becomes *Play = Yes* → Remaining dataset contains only positive examples:

Index	Outlook	Temperature	Humidity	Windy	Play
4	Rainy	Mild	High	FALSE	Yes
5	Rainy	Cool	Normal	FALSE	Yes
7	Overcast	Cool	Normal	TRUE	Yes
9	Sunny	Cool	Normal	FALSE	Yes
10	Rainy	Mild	Normal	FALSE	Yes
11	Sunny	Mild	Normal	TRUE	Yes
12	Overcast	Mild	High	TRUE	Yes
13	Overcast	Hot	Normal	FALSE	Yes
3	Overcast	Hot	High	FALSE	Yes

- **First Pattern: The Obvious One**

Check *Outlook = Overcast* → Covers: 4; All are *Yes*

→ Perfect rule immediately

Rule (3)

IF Outlook = Overcast → Play = Yes

→ Remove all the *Outlook = Overcast* samples.

Step 7: Continue Learning — Now It Gets Subtle

Remaining data is more mixed:

Index	Outlook	Temperature	Humidity	Windy	Play
4	Rainy	Mild	High	FALSE	Yes
5	Rainy	Cool	Normal	FALSE	Yes
9	Sunny	Cool	Normal	FALSE	Yes
10	Rainy	Mild	Normal	FALSE	Yes
11	Sunny	Mild	Normal	TRUE	Yes

Try: *Outlook = Sunny* → Mixed → not good enough

Add refinement *Humidity = Normal* → Covers: 2; All *Yes*

Rule (4)

IF Outlook = Sunny AND Humidity = Normal → Play = Yes

Remove these samples

Step 8: Final Pattern — The Last Piece

Remaining examples:

Index	Outlook	Temperature	Humidity	Windy	Play
4	Rainy	Mild	High	FALSE	Yes
5	Rainy	Cool	Normal	FALSE	Yes
10	Rainy	Mild	Normal	FALSE	Yes

→ All are Rainy cases

Check: *Windy* = *FALSE* → Covers: 3; All *Yes*

Rule (5)

IF Outlook = Rainy AND Windy = FALSE → Play = Yes

Final Rule Set — The Model's "Understanding"

After everything. After growing, pruning, removing, repeating. RIPPER ends with:

1. IF Outlook = Sunny AND Humidity = High → Play = No
2. IF Outlook = Rainy AND Windy = TRUE → Play = No
3. IF Outlook = Overcast → Play = Yes
4. IF Outlook = Sunny AND Humidity = Normal → Play = Yes
5. IF Outlook = Rainy AND Windy = FALSE → Play = Yes

Notice something subtle, the RIPPER rule set is **logically equivalent** to the Decision Tree we built earlier. But the way they learn is completely different:

- Decision Trees → **Top-down (divide-and-conquer)**
- RIPPER → **Bottom-up (separate-and-conquer)**

Same knowledge. Different thinking.

You might ask: *"Why not just use accuracy to choose rules?"*

Because accuracy can be misleading. A rule can be highly accurate, simply by covering very few examples → That's not learning. That's **avoiding mistakes**.

Instead, RIPPER uses **information gain**: It rewards rules that cover **many true positives** while avoiding negatives

So the goal is not just: "Be correct" → But: "Be correct **and useful**"

- Decision Trees → learn everything at once
- RIPPER → learns **one reliable pattern at a time**

And instead of chasing accuracy. It chases **meaningful coverage**. Because in real learning:

┆ *"A rule that explains more... is worth more."*

6. RIPPER Algorithm - The Pruning Phase

“Now, prove that you deserve to stay”

After growing rules, RIPPER does something most algorithms don't: It **questions every rule it just created**. Because a rule that looks perfect on training data might completely fail on unseen data.

The Pruning Metric

To evaluate a rule, RIPPER uses a simple but powerful idea:

- p : number of **correct predictions**
- n : number of **incorrect predictions**

Instead of raw accuracy, it uses a **smoothed estimate**:

$$\frac{p + 1}{p + n + 2}$$

Why This Formula? Because it fixes a critical problem: Rules that cover very few examples can look “perfect”. This formula prevents that by penalizing low coverage, avoiding overly optimistic scores

So RIPPER prefers rules that are: **Accurate AND Reliable**

Dataset Split — The Hidden Strength

RIPPER never evaluates rules on the same data it used to build them. It splits the data into:

- **Grow Set** → Build rules → *“Be expressive”*
- **Prune Set** → Test rules → *“Be realistic”*

Together: One creates rules... the other keeps them honest.

Evaluating the Rules

Now we test each rule on the **prune dataset**.

Rule	Condition	p	n	Score
(1)	Sunny & High → No	1	0	0.667
(2)	Rainy & Windy=TRUE → No	1	0	0.667
(3)	Overcast → Yes	3	0	0.800
(4)	Sunny & Normal → Yes	0	0	0.500
(5)	Rainy & Windy=FALSE → Yes	0	0	0.500

- **Rule (3) is the Strongest** → Covers multiple examples → All correct → Highest score: 0.8. This is exactly what RIPPER wants: Simple; General; Reliable
- **Rules (1) and (2)**: Each correctly classifies 1 example → No mistakes → Score: **0.667**. Good, but limited coverage.
- **Rules (4) and (5)**: Match **no examples** in prune set → Score defaults to: $\frac{1}{2} = 0.5$. This is intentional: “No evidence” = neutral confidence

Attempting Pruning — Does Simpler Mean Better?

Now RIPPER asks: "What if we remove some conditions?"

Example: Rule (4)

Original: **Sunny & Humidity = Normal** → **Yes** → **Score = 0.5**

Pruned version: **Sunny** → **Yes**

Now: Matches a negative example → $p' = 0, n' = 1$ → New score: $\frac{1}{3} \approx 0.333$ → Worse than before → **Pruning rejected**

Example: Rule (5)

Original: **Rainy & Windy = FALSE** → **Yes** → **Score = 0.5**

Pruned: **Rainy** → **Yes** : Again introduces error → score drops to 0.333 → **Pruning rejected**

After evaluating all rules: **No pruning improves performance**. So RIPPER keeps all rules **unchanged**.

This is where RIPPER shows its true strength: It **does not simplify blindly** → It simplifies **only when it improves generalization**. Because a simpler rule is not always a better rule. Only a rule that performs better on unseen data is worth keeping.

- Growing phase → "Find patterns"
- Pruning phase → "Test those patterns against reality"

And only the rules that survive both become the final model.

7. RIPPER Algorithm with Python

We typically implemented the Ripper Algorithm with the Wittgenstein library (but still require Scikit-Learn to split data)

***Wittgenstein** is a lightweight Python library designed specifically for **rule-based machine learning**, most notably implementations of the **RIPPER algorithm** (Repeated Incremental Pruning to Produce Error Reduction). It focuses on generating **human-readable IF-THEN rules** for classification, making it especially useful when interpretability is important (e.g., decision support systems, explainable AI).*

We use this toy dataset:

[play_tennis_ripper.csv](#)

```
import pandas as pd

# Load dataset
df = pd.read_csv("play_tennis_ripper.csv")

# Apply label encoding to categorical features
```

```
for i in df.columns:
    unique_columns = df[i].unique()
    df[i] = df[i].map({unique_columns[value]: value for value in range
(len(unique_columns))})
```

```
from sklearn.model_selection import train_test_split

# Split data
train, test = train_test_split(df, test_size=0.3, random_state=42)
```

```
import wittgenstein as lw

# Train RIPPER (with pruning ON by default)
ripper = lw.RIPPER()
ripper.fit(train, class_feat="Play", pos_class=1)

# Display rules
print(ripper.ruleset_)
```

```
[[Outlook=0^Windy=1] V [Outlook=2^Windy=1] V [Humidity=1^Outlook=0] V [Outlook=2^Humidity=1]]
```

```
# Predict on new data
predictions = ripper.predict(test)
# Evaluate accuracy
accuracy = (predictions == test["Play"]).mean()
print(f"Accuracy: {accuracy:.2f}")
```

Accuracy: 1.00

Summary

Rule-Based Classifiers learn by expressing knowledge as a set of human-readable rules:

IF condition → THEN prediction

Instead of modeling everything at once, they break problems into **clear, interpretable patterns**.

RIPPER is one of the most powerful rule-based algorithms because it follows a disciplined process:

- Learn rules sequentially (separate-and-conquer) → Grow rules using information gain →
- Prune rules using a validation set → Optimize and remove weak rules

But more importantly, RIPPER is not just about building rules. It is about: Learning → Questioning → Keeping only what truly works (Pruning).

RIPPER succeeds because it balances two forces: Expressiveness (finding patterns) and Generalization (avoiding overfitting)

And in the end. It doesn't try to explain everything at once. It builds understanding **one reliable rule at a time**.

References

<https://www.geeksforgeeks.org/machine-learning/ripper-algorithm/>

<https://www.quantbeckman.com/p/the-ripper-reckoning-extracting-rules>

<https://www.youtube.com/watch?v=fmZaVTpRO-Q>

<https://www.youtube.com/watch?v=wgyvJCGOSj8>

Next Section:

 [35. Ridge & Lasso Regression](#)